

```

// server.js - Refael Fashion | Unified Node.js + Express backend (no payment gat
// Run: npm i express && node server.js → open http://localhost:3000
'use strict';

const path = require('path');
const express = require('express');

const app = express();
app.use(express.json());

/* =====
 * 1) CATALOG + SIZE CONSTANTS (bundled in server state)
 * ===== */
const SIZE_GROUPS = { KIDS: 'kids', YOUTH: 'youth', SHOES: 'shoes', COMBINED: 'co
const range = (a, b) => Array.from({ length: b - a + 1 }, (_, i) => String(a + i)
const SIZES_BY_GROUP = {
  [SIZE_GROUPS.KIDS]: range(0, 7),      // ילדים 7-0
  [SIZE_GROUPS.YOUTH]: range(8, 22),    // נוער 22-8
  [SIZE_GROUPS.SHOES]: range(20, 46),   // נעליים 46-20
  [SIZE_GROUPS.COMBINED]: range(0, 22), // 22-0 חסולב
  [SIZE_GROUPS.ONE_SIZE]: ['ONE_SIZE'], // מידה אחת
};

const isOneSize = (g) => g === SIZE_GROUPS.ONE_SIZE;
const sizesFor = (g) => SIZES_BY_GROUP[g] || [];

const PRODUCTS = [
  { id: 'suit-kids',      name: 'חליפת ילדים',      category: 'חליפות',  pric
  { id: 'suit-youth',    name: 'חליפת נוער',        category: 'חליפות',  pric
  { id: 'linen-shirt-kids', name: 'מכופתרת פשתן ילד',    category: 'מכופתרות', pric
  { id: 'shirt-regular', name: 'מכופתרת רגילה',    category: 'מכופתרות', pric
  { id: 'linen-pants-kids', name: 'מכנס פשתן ילדים',    category: 'מכנסיים',  pric
  { id: 'linen-pants-youth', name: 'מכנס פשתן נוער',    category: 'מכנסיים',  pric
  { id: 'parade-pants-kids', name: 'מכנס ילדים פרדה',    category: 'מכנסיים',  pric
  { id: 'parade-pants-youth', name: 'מכנס נוער פרדה',    category: 'מכנסיים',  pric
  { id: 'vest',          name: 'וסט נוער וילדים',    category: 'וסטים',   pric
  { id: 'shoes',          name: 'נעליים',        category: 'אביזרים', pric
  { id: 'belt',           name: 'חגורות',        category: 'אביזרים', pric
  { id: 'bowtie',         name: 'פפיון',          category: 'אביזרים', pric
];

const CATALOG = PRODUCTS.map((p) => ({
  ...p, oneSize: isOneSize(p.sizeGroup), sizes: sizesFor(p.sizeGroup), stockStatu
}));

const productById = new Map(CATALOG.map((p) => [p.id, p]));

```

```

function isValidSize(product, size) {
  if (!product) return false;
  if (product.oneSize) return true;
  return sizesFor(product.sizeGroup).includes(String(size));
}

/* =====
* 2) SHIPPING (server-side source of truth)
* ===== */
const SHIPPING_METHODS = { COURIER: 'courier', SELF_PICKUP: 'self_pickup' };
const FREE_SHIPPING_THRESHOLD = 1500;
const FLAT_SHIPPING_FEE = 40;
const isValidMethod = (m) => Object.values(SHIPPING_METHODS).includes(m);

function calcShippingFee(method, subtotal) {
  if (method === SHIPPING_METHODS.SELF_PICKUP) return 0;
  if (Number(subtotal) >= FREE_SHIPPING_THRESHOLD) return 0;
  return FLAT_SHIPPING_FEE;
}

function amountToFreeShipping(method, subtotal) {
  if (method === SHIPPING_METHODS.SELF_PICKUP) return 0;
  const r = FREE_SHIPPING_THRESHOLD - Number(subtotal);
  return r > 0 ? Math.ceil(r) : 0;
}

function shippingMessage(method, subtotal) {
  if (method === SHIPPING_METHODS.SELF_PICKUP) return 'להחטודיו - ללא דמי משלוח';
  const r = amountToFreeShipping(method, subtotal);
  return r > 0 ? `למשלוח חיינם` : `זכאי/ת למשלוח חיינם - המשלוח עלינו ₪${r} עוד`;
}

const round2 = (n) => Math.round((Number(n) + Number.EPSILON) * 100) / 100;

// Build authoritative line items from a client cart; prices come from the catalog
function buildLineItems(cartItems) {
  if (!Array.isArray(cartItems) || cartItems.length === 0) return { error: 'ריקה' };
  const items = [];
  for (const raw of cartItems) {
    const product = productById.get(String(raw.productId));
    if (!product) return { error: `מוצר לא קיים: ${raw.productId}` };
    if (product.stockStatus === 'out_of_stock') return { error: `${product.name}` };
    const size = product.oneSize ? 'ONE_SIZE' : String(raw.size ?? '');
    if (!product.oneSize && !size) return { error: `יש לבחור מידה עבור: ${product.name}` };
    if (!isValidSize(product, size)) return { error: `מידה לא תקינה עבור: ${product.name}` };
    const quantity = Math.max(1, parseInt(raw.quantity, 10) || 1);
    items.push({ productId: product.id, name: product.name, size, quantity, price });
  }
  return { items };
}

```

```

function computeTotals(method, items) {
  const subtotal = items.reduce((s, it) => s + it.price * it.quantity, 0);
  const shipping = calcShippingFee(method, subtotal);
  return {
    subtotal: round2(subtotal),
    shipping: round2(shipping),
    grandTotal: round2(subtotal + shipping),
    remainingForFreeShipping: amountToFreeShipping(method, subtotal),
    shippingMessage: shippingMessage(method, subtotal),
  };
}

/* =====
* 3) VALIDATION
* ===== */
const ISRAELI_MOBILE = /^05\d{8}$/; // normalized: 10 digits, starts 05
const normalizePhone = (raw) => String(raw || '').replace(/\D/g, '');
const isValidEmail = (e) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/i.test(String(e || '').trim());

function validateCustomer(payload) {
  const errors = {};
  const c = payload.customer || {};
  const method = payload.shippingMethod;

  if (!String(c.firstName || '').trim()) errors.firstName = 'שם פרטי הוא שדה חובה';
  if (!String(c.lastName || '').trim()) errors.lastName = 'שם משפחה הוא שדה חובה';

  const phone = normalizePhone(c.phone);
  if (!phone) errors.phone = 'מספר טלפון הוא שדה חובה';
  else if (!ISRAELI_MOBILE.test(phone)) errors.phone = 'מספר נייד לא תקין (05 רחט)';

  if (!String(c.email || '').trim()) errors.email = 'אימייל הוא שדה חובה';
  else if (!isValidEmail(c.email)) errors.email = 'כתובת אימייל לא תקינה';

  if (!isValidMethod(method)) errors.shippingMethod = 'יש לבחור שיטת משלוח';

  // Address required ONLY for courier.
  if (method === SHIPPING_METHODS.COURIER) {
    if (!String(c.city || '').trim()) errors.city = 'עיר / יישוב הוא שדה חובה';
    if (!String(c.street || '').trim()) errors.street = 'רחוב הוא שדה חובה';
    if (!String(c.houseNumber || '').trim()) errors.houseNumber = 'ת.הוא שדה חובה';
  }

  const customer = {
    firstName: String(c.firstName || '').trim(),
    lastName: String(c.lastName || '').trim(),
    fullName: `${String(c.firstName || '').trim()} ${String(c.lastName || '').trim()}`
  };
}

```

```

    phone,
    email: String(c.email || '').trim(),
    notes: String(c.notes || '').trim(),
    ...(method === SHIPPING_METHODS.COURIER
      ? {
        city: String(c.city || '').trim(),
        street: String(c.street || '').trim(),
        houseNumber: String(c.houseNumber || '').trim(),
        entrance: String(c.entrance || '').trim(),
        floor: String(c.floor || '').trim(),
        apartment: String(c.apartment || '').trim(),
        zip: String(c.zip || '').trim(),
      }
      : {}),
  };
  return { ok: Object.keys(errors).length === 0, errors, customer };
}

// Payment scenario derived from shipping method (server-side).
function paymentFor(method) {
  return method === SHIPPING_METHODS.SELF_PICKUP
    ? { paymentMethod: 'studio_cash', status: 'pending_pickup_cash', label: 'סטודיו' }
    : { paymentMethod: 'courier_cash', status: 'pending_cash', label: 'זמן לשליח' };
}

/* =====
* 4) IN-MEMORY ORDER STORE (swap for a real DB later)
* ===== */
const orders = [];
let orderSeq = 1000;
const orderRepository = {
  create(order) {
    const created = { id: `RF-${++orderSeq}`, ...order };
    orders.push(created);
    return created;
  },
  getById(id) {
    return orders.find((o) => o.id === id) || null;
  },
};

/* =====
* 5) ENDPOINTS
* ===== */
app.get('/api/catalog', (_req, res) => res.json({ products: CATALOG }));

// POST /api/checkout/quote - live totals + milestone message

```

```

app.post('/api/checkout/quote', (req, res) => {
  try {
    const method = isValidMethod(req.body?.shippingMethod) ? req.body.shippingMet
    const built = buildLineItems(req.body?.items);
    if (built.error) return res.status(400).json({ error: built.error });
    const totals = computeTotals(method, built.items);
    return res.json({ shippingMethod: method, payment: paymentFor(method), ...tot
  } catch (err) {
    console.error('quote error', err.message);
    return res.status(500).json({ error: 'שגיאה בחישוב העגלה' });
  }
});

```

```

// POST /api/checkout/submit – validate, compute final price, create order
app.post('/api/checkout/submit', (req, res) => {
  try {
    const method = req.body?.shippingMethod;
    if (!isValidMethod(method)) return res.status(400).json({ error: 'שיטת משלוח'

    const { ok, errors, customer } = validateCustomer(req.body);
    if (!ok) return res.status(422).json({ error: 'טופס לא תקין', fields: errors

    const built = buildLineItems(req.body?.items);
    if (built.error) return res.status(400).json({ error: built.error });

    const totals = computeTotals(method, built.items);
    const pay = paymentFor(method);

    const order = orderRepository.create({
      status: pay.status,
      paymentMethod: pay.paymentMethod,
      paymentLabel: pay.label,
      shippingMethod: method,
      customer,
      items: built.items,
      subtotal: totals.subtotal,
      shipping: totals.shipping,
      grandTotal: totals.grandTotal,
      createdAt: new Date().toISOString(),
    });

    return res.status(201).json({
      ok: true,
      orderId: order.id,
      status: order.status,
      paymentLabel: order.paymentLabel,
      totals: { subtotal: totals.subtotal, shipping: totals.shipping, grandTotal:

```

```

    });
  } catch (err) {
    console.error('submit error', err.message);
    return res.status(500).json({ error: 'אירעה שגיאה ביצירת ההזמנה. נסו שוב.' });
  }
});

/* =====
 * 6) ADMIN (orders board + catalog/inventory editor)
 * Optional auth: set ADMIN_TOKEN in env to require an "x-admin-token" header.
 * For production, replace this with real authentication.
 * ===== */
const ORDER_STATUSES = ['pending_cash', 'pending_pickup_cash', 'completed', 'canceled'];
const STOCK_STATUSES = ['in_stock', 'out_of_stock'];

function adminAuth(req, res, next) {
  const required = process.env.ADMIN_TOKEN;
  if (required && req.get('x-admin-token') !== required) {
    return res.status(401).json({ error: 'גישת נדחתה' });
  }
  next();
}

// All submitted orders (newest first) for the admin board.
app.get('/api/admin/orders', adminAuth, (_req, res) => {
  res.json({ orders: orders.slice().reverse() });
});

// Update an order status.
app.post('/api/admin/orders/:id/status', adminAuth, (req, res) => {
  const order = orderRepository.getById(req.params.id);
  if (!order) return res.status(404).json({ error: 'הזמנה לא נמצאה' });
  const status = req.body?.status;
  if (!ORDER_STATUSES.includes(status)) return res.status(400).json({ error: 'ק"ל' });
  order.status = status;
  order.updatedAt = new Date().toISOString();
  res.json({ ok: true, order });
});

// Update a master product's price and/or stock status (in-memory).
app.post('/api/admin/products/:id', adminAuth, (req, res) => {
  const product = productById.get(String(req.params.id));
  if (!product) return res.status(404).json({ error: 'מוצר לא קיים' });

  const { price, stockStatus } = req.body || {};
  if (price !== undefined) {
    const p = Number(price);

```

```

    if (!Number.isFinite(p) || p < 0) return res.status(400).json({ error: 'תקין
    product.price = round2(p);
  }
  if (stockStatus !== undefined) {
    if (!STOCK_STATUSES.includes(stockStatus)) return res.status(400).json({ erro
    product.stockStatus = stockStatus;
  }
  res.json({ ok: true, product });
});

/* =====
* 7) SERVE THE FRONTEND
* ===== */
app.get(['/', '/checkout'], (_req, res) => res.sendFile(path.join(__dirname, 'che
app.get('/admin', (_req, res) => res.sendFile(path.join(__dirname, 'admin.html'))

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Refael Fashion server → http://localhost:${PO

```